

参数选择方法分析

基于刷新+重载的RSA缓存定时攻击

周平¹、王涛¹、李光²、张帆³、赵新杰²

中国石家庄050003 军械工程学院信息工程系

² 中国北京100083 北方电子设备研究所

³Department 信息科学与电气工程, 浙江大学, 杭州, 310027, 中国

摘要: flush+reload攻击作为一种新型缓存定时攻击手段, 其攻击机制包含三个关键要素: 监控指令、阈值设定及等待间隔。然而现有研究鲜少深入探讨这些要素如何影响系统性能及其作用机理。本文系统研究了这三个参数的影响规律, 并提出优化参数选择的完整方法。基于必要充分条件, 我们构建了监控指令的筛选规则体系; 同时运用贝叶斯二元信号检测原理, 提出了最优阈值的选取方案。此外, 我们还建立了监控时间序列模型, 并明确了最优等待间隔的计算方法。

对二进制平方乘法算法实现的RSA算法进行了大量实验, 结果表明, 全密钥恢复的平均成功率达到了89.67%。

关键词: 侧信道攻击; 缓存定时攻击; RSA; 平方乘算法; 指数运算

一、引言

侧信道攻击[1][2]是一种针对密码系统实现的攻击类型。此类攻击利用物理特性进行攻击。

在加密和解密过程中, 硬件平台会泄露诸如时间、声音、功耗、电磁辐射等信息, 这些信息能够揭示算法的内部状态, 从而推断出密钥的相关知识。

缓存定时攻击[3]属于侧信道攻击的一种。其攻击原理是利用缓存访问与非缓存访问之间的时间差作为信息泄露的源头。攻击者通常会利用间谍进程将缓存设置为已知状态, 并通过监控状态变化来获取目标进程的缓存访问信息。自缓存定时攻击提出以来, 该技术已被用于破解多种公钥加密算法和分组密码, 例如RSA[4]、DSA[6]、ElGamal[14]以及AES[7][15][16][17]。

针对RSA的现有缓存时序攻击主要分为两类。第一类攻击通过监控特定指令实现[5][8]。间谍进程的虚拟指令会与目标进程的特定指令精确映射到同一级联缓存(L1 Cache)位置, 从而在虚拟指令与目标指令之间制造冲突, 使攻击者能够检测到目标进程执行特定指令的行为。另一类攻击则通过监控整个级联缓存[4][6], 攻击者可将整个一级缓存(L1 Cache)填满

通过生成虚拟指令来模拟容量缺失，从而推断出受害者执行的操作，因为这些操作具有不同的缓存访问模式。

然而，此类攻击依赖于间谍进程与目标进程共享同一 L1Cache。因此，攻击单核处理器上实现的RSA算法是可行的。对于多核处理器，间谍进程与目标进程可能被分配到不同核心，导致一级缓存无法共享。

为解决这一问题，尤瓦尔·亚罗姆和卡特里娜·福克纳[9]提出了一种名为“刷新+重载攻击”的L3缓存定时攻击方法。该技术基于间谍进程与目标进程共享物理内存页的特性：间谍进程会从缓存各层级中刷新目标进程在共享内存页中的特定指令，随后将这些指令重新加载到缓存并测量耗时，从而实现对目标进程访问特定指令的监控。该攻击方法曾被用于从GnuPG中提取私钥，成功恢复了RSA私钥98%的比特位。

随后，Yarom和Benger[18]将该方法应用于OpenSSL对二进制域的 ECDSA 实现。NaomiBenger等人[19]将flush+reload攻击与格技术相结合，攻击了wNAF点乘算法，并在获得少于256个签名后，成功重构了256位椭圆曲线的密钥。

然而，现有研究鲜少分析监测指令、等待间隔及阈值的影响。如何确定这些参数的最优值也未得到充分探讨。基于[9]，本文对上述三个问题进行了详细分析，并进行了实验验证。

首先，我们分析了受害进程的指令访问与操作执行之间的关系。随后，我们提出了基于必要充分条件的监控指令选择规则。

此外，对OpenSSL库的指数运算代码进行了分析，并选择了适当的监控指令。

同时，提出了一种基于贝叶斯二元信号检测原理的阈值选择方法，通过计算错误判定的最小代价，找到最优阈值并计算其值，实验表明，使用最优阈值，攻击成功率提高，误判率降低。

同时，给出了监控的时间序列模型和计算等待间隔时间的公式，实验结果表明，当等待间隔时间在理论计算值范围内时，受害访问的捕获率在62.82%以上。

在所选最优参数下，对基于平方乘算法的RSA算法进行了大量实验，结果表明，所提攻击的成功率在86%~95%之间，而文献[9]中所选参数的成功率仅为64%。

本文结构安排如下：第二节介绍RSA算法的背景知识；第三节对典型的“刷新+重载”攻击进行综述与讨论；第四节提出参数选择方法；第五节展示实验结果并进行讨论；最后，第六节对全文进行总结。

二、RSA 加密及其实施

2.1 RSA加密

RSA[13]是目前最广泛使用的公钥密码体制，它可用于加密和数字签名，RSA公钥加密的密钥生成步骤如下：

- 生成两个大随机素数 p 和 q 。
- 计算 $N=pq$ 和 $\Phi = (p-1) \cdot (q-1)$ 。
- 随机选取一个整数 e （满足 $1 < e < \Phi$ ），

本文研究了监控指令的三个参数——阈值、等待间隔和监控指令的选取方法对监控指令的影响，提出了基于必要充分条件的监控指令选取的完整规则。

算法1 平方乘法算法

输入：底 x ，模 N ，指数 $e=(e_1e_2\cdots e_t)$ 的平方， $e_1=1$ 。

输出： $y=x^e \bmod N$

计算：

$y=1$

for i from t to 0 {

$y=\text{平方}(y)$

$y=\text{ModReduce}(y, N)$

if $e_i=1$, then {

$y=\text{Mult}(y, x)$

$y=\text{ModReduce}(y, N)$

}

f

返回(y)

其中 $\gcd(e, \varphi) = 1$ 。

·求唯一整数 d (满足 $1 < d < \varphi$)，使得 $ed=1 \pmod{\varphi}$

- 公钥为 (e, N) ，私钥为 d 。

设 m 为消息， c 为密文，加密时发送方应

做如下工作：

·将消息表示为区间 $[0, n-1]$ 内的整数 m 。

·计算 $c = m^e \bmod N$ 。

为从 c 中恢复明文 m ，接收方需执行以下步骤：

·计算 $m = c^d \bmod N$

2.2 平方乘法算法

RSA算法中最重要的算术运算是模指数运算。平方乘法算法是二进制指数运算的基本方法，首先将指数表示为二进制形式：

$$e = \sum_{i=0}^t 2^i * e_i$$
然后通过扫描指数的比特进行模指数运算。指数比特可从LSB向MSB方向扫描，或反向扫描。这两种方法分别称为‘右向左’和‘左向右’指数运算。

本文不作一般性限制，重点讨论第二种情况。

在“左到右”方法中，每次迭代首先执行平方运算和模约简运算。根据当前指数位值的不同，随后可执行乘法运算和模约简运算。该方法的具体实现详见算法1。

我们发现，二进制平方乘法算法中执行的操作与指数位相关。当当前指数位为0时，会执行一次平方操作和一次归约操作；而当指数位为1时，则会额外执行乘法和归约操作。因此，攻击者只需追踪平方乘法指数运算的执行过程，就能精确还原指数值。

H. 洗牌+重装攻击

3.1 攻击原理

flush+reload 是最近提出的一种

针对多核处理器上公钥密码体制实现的缓存攻击。

该技术通过监控和追踪间谍进程对共享内存页中指令的访问行为，实现对目标进程的动态分析。间谍进程仅需对共享内存页进行读取操作，即可获取目标进程执行指令的详细信息。通过分析目标进程是否访问这些内存页中的特定指令（或内存行），间谍进程能够推断出目标进程的行为模式。

每轮监测分为三个阶段。

第一阶段：间谍进程通过`clflush`指令清除缓存中所有层级的监控指令。

第二阶段：间谍进程等待一段时间。在此期间，受害进程可能在共享内存页面中加载被监控的指令。

第三阶段：间谍程序重新加载监控指令并测量所需时间。

若受害进程未访问该数据

在第二阶段监控指令时，需要从内存重新加载这些指令，因此耗时较长。若已缓存，则加载速度会显著加快。通过迭代执行这三个步骤，间谍进程可逐步掌握目标进程执行的监控指令序列。

3.2 再次发作

在flush+reload攻击方法中，三个特别重要的参数对攻击成功率至关重要。然而，现有文献均未详细讨论和分析这些参数的具体作用机制及其合理取值方法。

首要问题在于如何选择监控指令。间谍程序通过捕获目标进程对监控指令的访问行为，推断出解密过程中执行的操作序列（平方运算、乘法运算和模运算）。每个监控指令都应与其对应的操作相对应，同时需确保间谍程序能以足够高的概率捕获目标进程的访问行为。尤瓦尔·亚罗姆和卡特里娜·福克纳[9]曾指出“为提高探测器捕获访问行为的概率，我们选择了计算过程中频繁执行的内存行”。然而我们发现，这一原则并不能保证监控指令的访问行为与实际操作之间存在对应关系。若选择不当，可能导致操作判定出现偏差。

第二个问题是如何确定重载时间的最优阈值。该阈值用于与重载时间进行对比，以判断被监控的指令是从内存还是缓存中重新加载的。最优阈值可降低误判率。文献[9]将阈值设定为120个周期，但我们的实验发现该阈值并不理想。

第三个问题是如何选择等待间隔。在第二阶段，监测的指令-

如果等待时间设置过长，攻击者可能多次访问内存，导致监控设备无法准确识别。但若等待时间过短，当攻击者与受害者内存访问发生重叠时，监控设备就会错过访问机会——因为此时的重装时间会远超正常值。文献[9]将等待时间设定为2048个周期，但未说明具体原因。尽管有观点指出“选择时隙长度需要在攻击成功率与漏检率之间进行权衡” [18]，但等待时间是否存在上下限限制仍是一个悬而未决的问题。

四、刷+重载攻击的参数选择方法

4.1 基于必要充分条件的监控指令选择规则

在攻击的每一轮中，间谍进程会监控平方、乘法和模约简操作中的每条指令。

为便于表述，设addr为被监控指令的地址。其中addr、addr和addr分别表示平方运算、乘法运算和模约简运算的监控指令。A表示受害者程序中所有指令地址的集合，m、r、e分别对应平方运算、乘法运算、模约简运算及其他操作，而S、M、R、E则分别表示这些操作对应的地址集合。设b为缓存块的大小（单位：字节）。显然，我们有 $A=\sum_{i=1}^n a_i$ ，且该等式的右侧可能不为空集。

直接从测量结果中可以判断被监控指令是从Cache还是从内存中重新加载的，即命题p的真值：

命题p:addr，在等待间隔期间被缓存。（i=s, m, r）

攻击者需要从命题p中推断命题q的真值：

命题 q : 执行操作 i 。 ($i = s, m, r$)

根据攻击理论, 命题 p 和命题 q 必须同时为真或为假, 即若 p 则 q , 若非 p 则非 q 。因此, 应选择满足 p 与 q 互为充分必要条件的地址。

$$p \xleftrightarrow{addr_i} q, q \xleftrightarrow{addr_i} p$$

为确保在等待间隔 ($q \rightarrow p$) 期间执行操作 i 时 $addr$ 被缓存, $addr$ 应符合以下规则:

规则1: 在执行操作 i 期间, 必须由受害进程访问 $addr$:

$$addr_i \in S, addr_m \in M, addr_r \in R$$

为确保在等待间隔期 ($p \rightarrow q$) 未执行操作 i 时 $addr$ 不被缓存, $addr$ 应符合以下规则:

规则2: 除 i 操作外, 执行任何操作期间, 受害进程不得访问 $addr$ 。

$$地址, MURUE, 地址 \in SURUE, \\ addr, eSUMUE$$

规则3: 其他操作的执行不应导致 $addr$, s 被间接缓存:

$$[addr, -2b, addr, +2b] \in MURUE, \\ [addr, -2b, addr_m + 2b] \in SURUE, \\ [addr, -2b, addr, +2b] \in SUMUE.$$

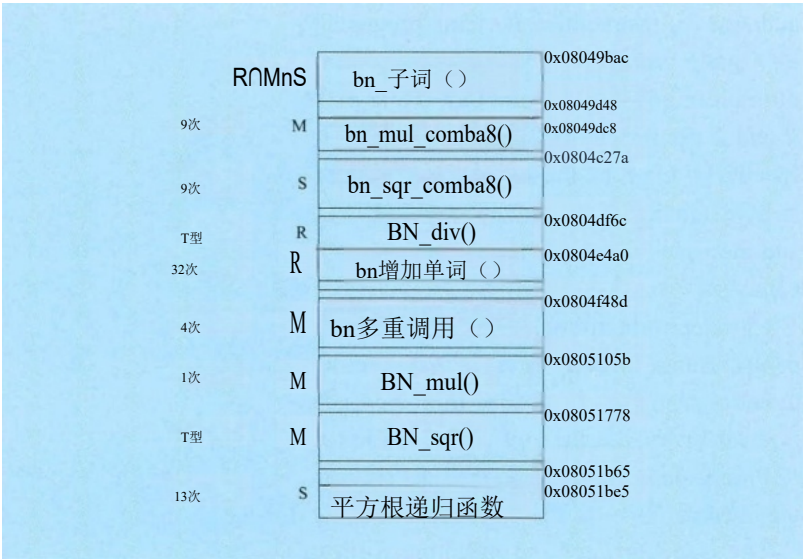


图1 地址分配

有两个原因使得 $addr$ 、需要间接缓存。第一个原因是指令和数据是以块为单位缓存的。如果 $addr$ 和 $addr$ 映射到同一个缓存块中, 那么由于执行操作 j , $addr$ 将与 $addr$ 一起被加载到缓存中。因此, 应确保在 $[addr-b, addr+b]$ 范围内不存在其他操作的指令。另一个原因是缓存预取。某些处理器会在内存中首次访问某个2字节缓存块时, 预取该块的另一对2字节缓存块。因此, 其他操作的指令不应位于 $[addr-2b, addr+2b]$ 范围内。前者应谨慎避免, 后者则取决于处理器的特性。

此外, 为了提高捕获机会并增强抗噪能力, $addr$ 应尽量遵循以下规则:

规则4: $addr$, 应该被访问的次数比其他任何地址都多。

$$n(addr_i) = \max n(addr), addr \in \{x | p \rightarrow q\}, (i = s, m, r)$$

其中 $n(addr)$ 表示访问 $addr$ 的时间。

在我们的实验中, 我们使用 OpenSSL 库实现了平方乘算法。图1展示了指数运算涉及的主要函数及其对应的地址。

在图1左侧, 标注了指令所属集合及各操作中访问时间。根据上述规则, 将位于 $addr, =0x08051be5$, $addr_m=0x08049dc8$, $addr, =0x0804e4a0$ 的指令选为监控目标。需注意, 根据规则3, 地址寄存器和地址寄存器的地址分别从函数首地址起算128字节 (即两个缓存块的大小)。此外, 尽管函数 $bn_sub_words()$ 被调用超过32次, 但根据规则2无法将其指令纳入监控范围, 因为该函数在执行所有三个操作时都会被调用。

4.2 基于二元信号贝叶斯检验原理的时间阈值选择

时间阈值用于判断监控指令是从缓存还是内存重新加载的。若加载时间超过阈值，则默认该指令是从内存而非缓存加载的。由此可推断，受害进程在间谍进程重新加载前已访问该指令。

由于其他进程的干扰和存储过程中的随机因素，监控程序只能在噪声环境中测量重装时间。噪声的存在可能导致两种判定错误：第一种是将内存重装误判为缓存重装，导致误判被监控指令被受害者访问；第二种是将缓存重装误判为内存重装，从而漏判受害者进程的访问行为。因此，选择合适的阈值至关重要，以最大限度降低误判的影响。

文献[9]未对不同阈值对私钥恢复或操作序列的影响进行分析。该文献的阈值选择方法如下：首先，主要在[33,49]和[200,300]区间内评估内存与L1缓存的加载时间。由于L3缓存的访问时间比L1缓存长22~39个周期，因此根据实测结果和英特尔手册，将阈值设定为120个周期。

4.2.1 贝叶斯原理与平均成本

本文采用二元信号贝叶斯检测原理选择阈值，该原理以最小平均成本为准则，通过统计检验确定最优信号状态。在本场景中，重载有两种状态：被监控指令从高速缓存重载（设为假设 H_0 ）；被监控命令从内存重载（设为假设 H_1 ）。由此可得出四种判定结果，记为 (H_{ij}) ($i, j=0,1$)，其中 $H(j=0$,

1)当条件成立时，即 H_i ($0,1$) 成立的结果。对应的判定概率表示为 $P(H_{ij})$ ($i, j=0,1$)。

确定概率的计算公式为：

$$P(H_i|H_j) = \int_{R_i} p(x|H_j)dx \quad i, j = 0, 1 \quad (4-1)$$

该公式中， $p(x|H_0)$ 表示缓存测得的重载时间概率密度函数； $p(x|H_1)$ 表示内存测得的重载时间概率密度函数。 R 表示重载时间观测空间中被阈值 η 划分的两个子空间。具体而言，当测得值小于阈值 η 时，数据落入空间 R_0 ，验证假设 H_0 ；当测得值大于阈值 η 时，数据落入空间 R_1 ，验证假设 H_1 。

对于四种判定场景，当 $i=j$ 时判定正确，否则判定错误。成本因子 c ；($i, j=0,1$) 用于表示每个判定场景的成本，即当 H ($=0,1$) 为真时，判定 H_i ($i=0,1$) 成立的成本。成本因子的约束条件为 $c \geq 0$, $j=0,1$; $c_{ij} \geq c_i$, $j=0,1$, $i \neq j$ 。

随后，平均成本为 $C = P(H_0)[c_{00}P(H_0|H_0) + c_{10}P(H_1|H_0)] + P(H_1)[c_{11}P(H_1|H_1) + c_{01}P(H_0|H_1)]$ (4-2)

根据贝叶斯检验原理，最优 η 使平均成本 C 最小化。

4.2.2 成本因素

在本场景中，四种判定结果对攻击的影响各不相同。当判定正确时，无需付出代价，因此 $c_{00}=0$, $c_{11}=0$ 。若判定错误，则 $c_{10} \gg c_{01} > 0$ 。这是因为每次操作中，被监控指令会被多次执行（例如在计算模约简时，被监控指令会被执行32次），多次检测这些指令足以准确判断受害进程执行的计算。因此， $(H_i|H_0)$ 次判定的成本并不显著。另一方面，一旦错误地将内存中的重装指令判定为来自缓存，

错误可能导致受害者产生错误推测。尽管密钥恢复算法已考虑错误因素，但私钥恢复的效率和成功率仍会大幅降低。因此， $(H_0 | H_1)$ 判定的成本将明显高于 $(H_1 | H_0)$ 判定的成本。本文设定 $c_{01}=100$ ， $c_{10}=10$ 。需注意，选择 c_{01} and c_{10} 是为了反映两种判定结果的相对成本。只要满足 $C_{01} > c_{10} > 0$ 的条件，后续分析将得出相同结果。

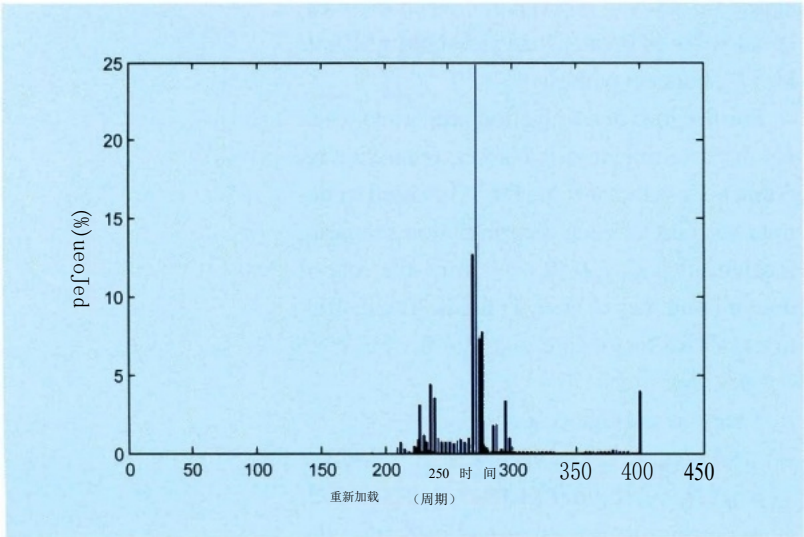


图2 内存重装时间分布

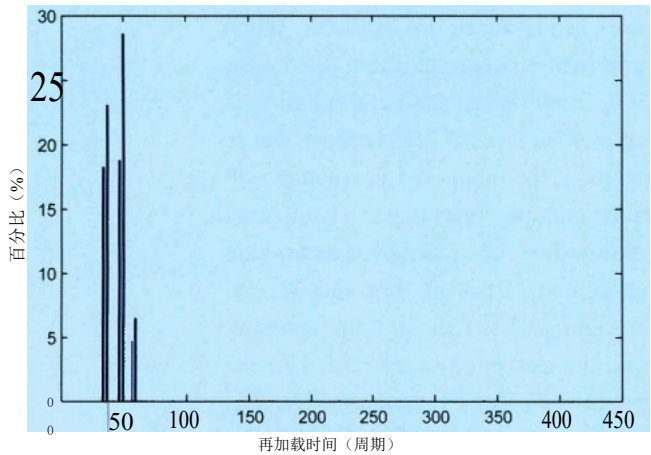


图3 缓存重载时间分布

4.2.3 先验概率与概率密度函数

由于受害进程在某一时刻仅能执行一个操作，因此三个被监控的指令中仅有一个会从高速缓存中重新加载。因此，两种判定假设的先验概率分别为 $P(H_0)=1/3$ 和 $P(H_1)=2/3$ 。

我们采用监控指令的内存与缓存重载时间分布作为计算平均成本的概率密度函数。

为测量监控指令从内存的重新加载时间，本研究未执行解密操作，仅运行间谍进程。通过这种方式，每次监控指令被加载到缓存后，都会被间谍进程清除，从而确保该指令在下次加载时会从内存重新加载。共测量了300,000次内存重新加载时间，其分布情况如图2所示。

图2中，横轴和纵轴分别表示重装时间与样本占总样本量的百分比。为使该图比例恰当，异常值已截断。从图2可见，所有内存重装时间均超过189个周期。实际上，大多数内存重装时间大于206个周期。

为获取监控指令的缓存重载次数，我们从监控程序中移除了`clflush`指令。通过这种方式，监控指令将始终从缓存中重载。共测量到300,000次缓存重载次数，其分布情况如图3所示。

从图3可以看出，大多数缓存重装时间分布在33~59个周期内，但也有部分样本分布在60~400个周期，超过了内存访问的最小时间，而且不同缓存层次的访问时间也从图中可以观察到。

4.2.4 最佳阈值的计算

在确定成本因素后，先验

假设概率与重载时间的概率密度函数，平均成本是阈值 η 的函数。通过 MATLAB，不同阈值下的平均成本如图4所示。

图4中，横轴表示阈值 η ，纵轴表示平均成本（为便于展示，结果已对数化）。从图4可明显看出， η 的最优值为170~189个周期，此时平均成本最低。

4.3 监控时间序列模型及等待时间间隔选择

在监控过程中，每轮监控操作分为刷新、等待和重新加载三个阶段。监控开始时，被监控指令会被监控进程从缓存中刷新。经过特定时间间隔的等待后，监控进程会重新加载该指令并记录加载时间。通过分析加载时间的长短，可以判断目标进程是否在等待期间访问了被监控指令。文献[9]将等待时间间隔设定为2048个时钟周期，但未对此进行详细说明。

本文认为，等待间隔对攻击的影响主要体现在两个方面：若等待间隔较长，被监控指令可能已被多次执行，或同一间隔内执行了多个不同的监控指令，这些指令无法被间谍进程区分，导致“分辨率”降低；若等待间隔较短，则频繁的刷新与重载操作更易与访问冲突。

受害者进程导致测量值剧烈波动，进而影响对受害者访问的捕获。

为研究等待间隔的范围，设twai为等待间隔，tprobe为每次监测操作所用时间（包括 冲洗、重新装载、测量记录等）。设trdsc表示计时器记录时间所用时间，tmemory和 tL3_Cache分别表示指令从内存和L3缓存的重新加载时间。设 ΔT_{target} 表示待区分的两个相邻被监控指令之间的执行时间间隔。所有变量均以周期为单位。

监控的时间序列模型如图5所示。

对于每个特定的监控指令，从刷新到下一次重装的时间包括等待间隔twait、另外两个监控任务2*tprobe的时间以及计时器指令rdtsc的时间成本trdise。基于时间序列模型

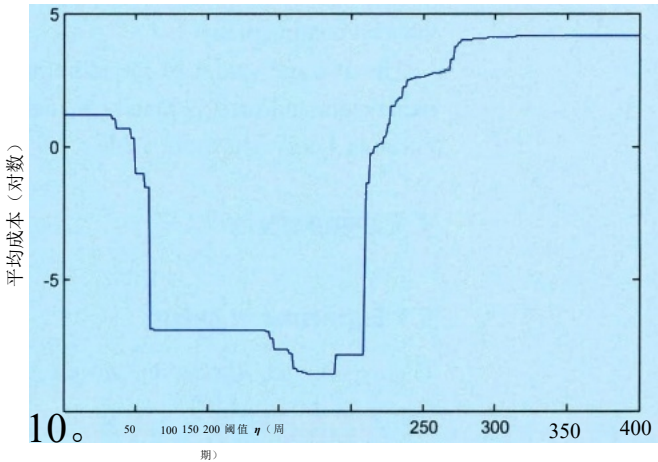


图4 不同阈值下的平均成本

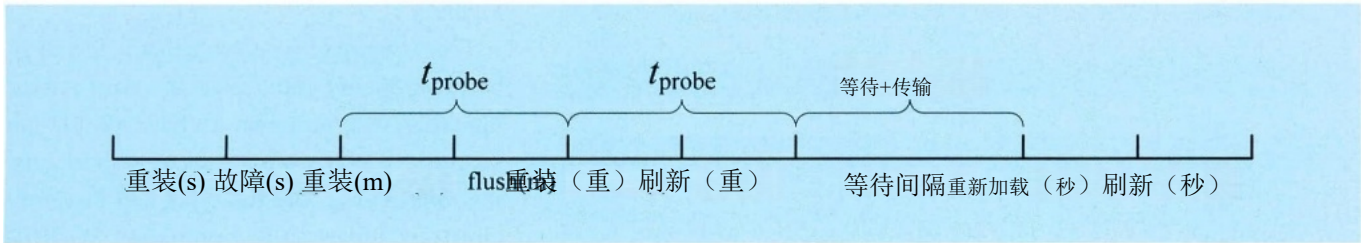


图5 监测时间序列模型

监测、等待间隔 t_{wait} 应满足：

$$t_{memory-L3_Cache} \leq t_{wai} + 2 * t_{probe} + t_{rdise} \quad \triangle \text{坦格特 (4-3)}$$

在flush+reload攻击中，攻击者需区分重载数据源自内存还是缓存。两轮监控间隔（即 $t_{wait} + 2 * t_{probe} + t_{atse}$ ）应至少大于被监控指令的加载时间。

L3 藏物处 从... 记忆，即 $t_{memory-L3_Cache}$ 。
否则，当监控进程尚未完成将监控指令从内存加载到L3缓存的操作时，监控进程就会开始重新加载和测量。由于不同进程访问同一物理地址所导致的冲突，会使得测量的重新加载时间产生偏差。

同时，两次监测间隔应确保访问行为不会在相同等待区间内发生。在本案例中，等待时间需小于每次被监测指令执行间隔的时间，同时也要小于同一计算中最后一次被监测指令执行与下一次计算中首次被监测指令执行之间的时间间隔。

上述参数的平均值及根据公式4-3计算的 t 值范围如表I所示。

五、实验

5.1 实验设置

该实验在ThinkCentre台式机上进行，其特征为

采用Intel i5-3470处理器，4GB DDR3-1600内存，Fedora 18操作系统，采用RSA算法，二进制平方乘法算法，平方乘法模约简算法由OpenSSL实现，每次解密使用随机生成的1024位模数和密钥，解密时选择的监控指令与第4.1节相同，监控地址由Eclipse调试工具获取。

在攻击中，如果连续两次以上 $addr$ 的重装时间小于阈值，则认为执行了操作 i ，成功的攻击意味着1024位被完全准确地恢复，没有耗尽搜索。

5.2 选择阈值

在实验中，我们采用操作判定错误率和攻击成功率来评估阈值的影响，并验证第4.2节的结论。操作判定错误率是指解密轮次中错误猜测操作数与实际执行操作总数的比值。错误猜测包括实际发生操作的误判以及从未发生操作的误判。

实验中，等待间隔设置为
[
0,50,100,150,200,250,300,400,600,800,1024,1536,2048,3072,4096
]个周期。每个间隔下，我们执行攻击100次并记录时间数据。随后采用不同阈值进行数据处理和密钥恢复。图6展示了操作判定错误率、攻击成功率与阈值之间的关系。

从图6可以看出，在80至160次循环范围内，随着阈值 η 的增加，操作判定的错误率下降，而攻击的成功率上升。这是因为错误判定概率 $P(H_1 | H_0)$ 降低所致。特别值得注意的是，错误率呈现

表 I 时间参数值与最优 t_{nai} 边界

参数	平均值 周期	最优 t_{wi} 的边界 周期
麦莫里	285.31	
三级高速缓存	57.74	
鼻子	512.31	
双极型	72.98	
$\triangle T_{mu}^{(5)}$	2059.53	<961.93
$\triangle T_{mge}^{(m)}$	3370.33	<2272.73
$T_m^{(6)}$	1476.04	<378.44
$\triangle T$ （操作间）	1670.40	<572.8

在120至160周期范围内出现急剧下降，而成功率在140至160周期范围内显著提升。这些特性源于Cache的重载时间测量值大多接近150周期。

此外，错误率趋于稳定，在180至200的范围内波动于1.78%至1.91%之间。当 $\eta = 180$ 时，错误率降至最低值1.78%。攻击成功率在84.47%至88.4%之间保持稳定，当 $\eta = 180$ 时达到峰值88.4%。该区间内， $P(H_0|H_0)$ 和 $P(H_1|H_1)$ 均处于较小范围。

攻击成功率急剧下降而错误率急剧上升的现象，归因于 η 超过内存重装时间最小值后， $P(H_0|H_1)$ 值的增加。

需注意阈值 η 在160至200之间具有相对较好的评估效果，其最优值为180。这些数值与第4.2节的结论一致。

5.3 选择等待间隔

本节采用测量标准差、捕获率及重叠率来评估等待间隔的影响，并验证第4.2节的结论。具体而言，测量标准差用于确定最优等待间隔 t_{wait} 的下限，而捕获率与重叠率则用于确定最优等待间隔 t_{wait} 的上限。

通常情况下，重装时间是稳定的。若出现波动，则表明刷新与重装过于频繁，从而干扰了被监控指令的访问。不同等待时间下的测量标准偏差如图7所示。

显然，随着等待间隔时间（wait）的增加，测量值的标准差（mea-）

当等待时间小于200时，测得的重装时间明显不稳定，可能导致数据处理时的判定错误。

捕获率是指实际捕获的受害者访问次数与受害者实际访问次数的比值。在较长的等待间隔期间，被监控的指令可能已被执行多次，导致捕获率下降。图8展示了不同等待值对捕获率的影响。

垂直虚线表示根据计算得出的最佳间隔时间边界

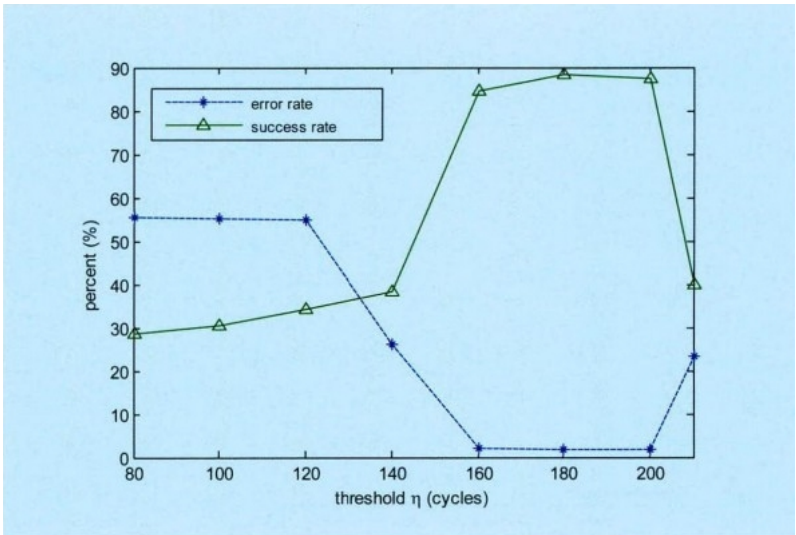


图6 不同阈值攻击的错误率与成功率

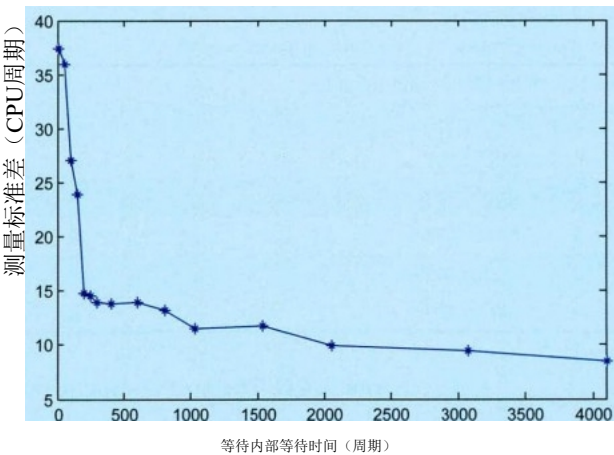


图7 不同等待间隔测量的标准偏差

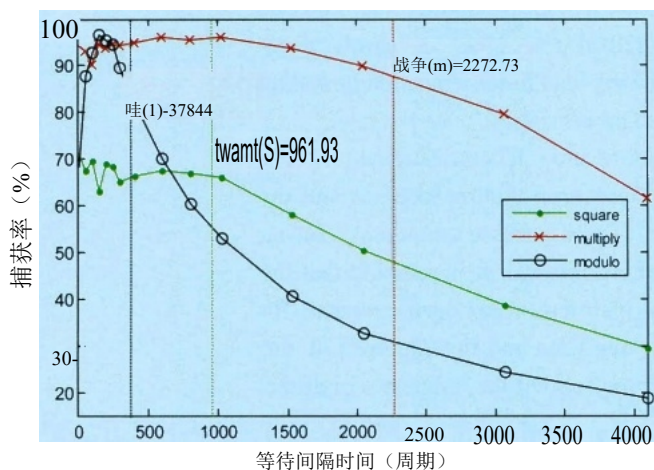


图8 不同等待间隔下的捕获率

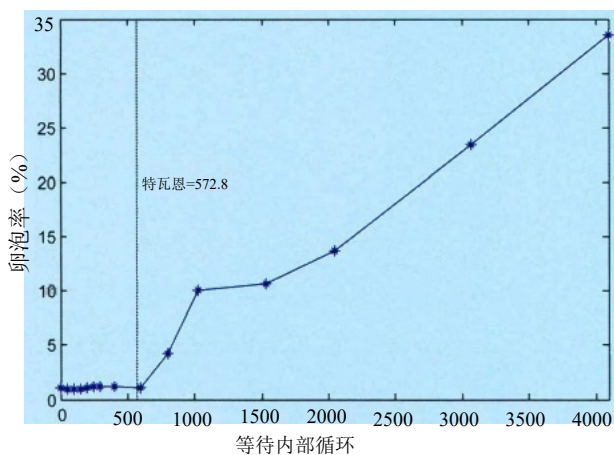


图9 不同等待间隔下的重叠率

表 II 相应于操作的监测轮次

监测轮次数量	操作
1317-1334	S
1337-1361	R
1361-1394	M
1394-1427	R
1428-1443	S
1447-1479	R

公式4-3. 总捕获率随等待时间增加而降低。当等待时间超过计算得出的最佳值边界后，捕获率急剧下降。

最佳值时，捕获率维持在62.82%以上。

此外，较长的等待间隔会导致不同操作之间出现重叠，从而造成操作难以区分，重叠率是重叠操作数与解密轮次中执行操作数的比值，等待间隔与重叠率的关系如图9所示。

从图9可以看出，当等待间隔 $twai > 600$ 时，重叠率急剧增加，实验结果与计算得到的最优等待间隔上限值非常接近，即 < 572.8 。

需注意，上述结果与第4.3节的结论一致。根据前述实验，后续实验将设置等待间隔为200~600个周期进行。

5.4 攻击实验

在攻击实验中，我们设置等待时间 $twait=200$ ， $\eta=180$ 。图10展示了某次解密操作期间的部分测量时间。

图10中，横轴表示监控轮次，纵轴表示重载时间，虚线为阈值。可以看出，当重载时间小于阈值时，被监控指令将从缓存中重新加载，从而确定此时受害进程执行的操作类型。

表II显示了与每次手术相对应的监测轮次：

在第1361轮模约简计算中，乘法运算与加法运算存在重叠；而在第1434轮中，平方运算中混入了错误判定，但操作序列仍能清晰辨识。这是因为所选监控指令在每次运算中频繁执行，从而提升了容错能力。

根据第5.2节和第5.3节的实验结果，我们设定 $160 < twait < 200$ 且 $200 < \eta < 600$ 。针对每个 $(twait, n)$ 组合，我们执行了100次攻击，每次攻击

我们使用了随机生成的1024位密钥。攻击成功率如表III所示：

在最优参数下，攻击成功率在86%~95%之间，平均为89.67%，而[9]所选参数下的攻击成功率仅为64%。

六、结论

本文深入分析了flush+reload攻击的三个关键问题，包括监控指令的选择、阈值设定及等待间隔的确定。我们基于必要充分条件提出了完整的监控指令选择规则，确保监控指令的重载时间能真实反映目标进程的实际操作。基于贝叶斯二进制信号检测原理，我们创新性地提出了最优阈值选择方法。实验结果表明，阈值应小于但接近内存加载时间的下限。研究建立了监控的时间序列模型，并提供了计算最优等待间隔的算法。总体而言，较短的等待间隔能提供更高分辨率，但若等待间隔过短可能导致重载时间测量不稳定。通过采用二进制平方乘法算法的RSA解密实验验证了该方法，平均成功率达到了89%。67%，优于文献[9]中64%的参数值。

需要说明的是，本文提出的方法通过简单调整细节，同样适用于RSA等公钥加密算法的多种实现方案。例如文献[18]中，由于两种错误判定的成本相同，在计算最优阈值时将成本因子 `coshould` 设定为 `clo`，这与我们的研究场景存在显著差异。

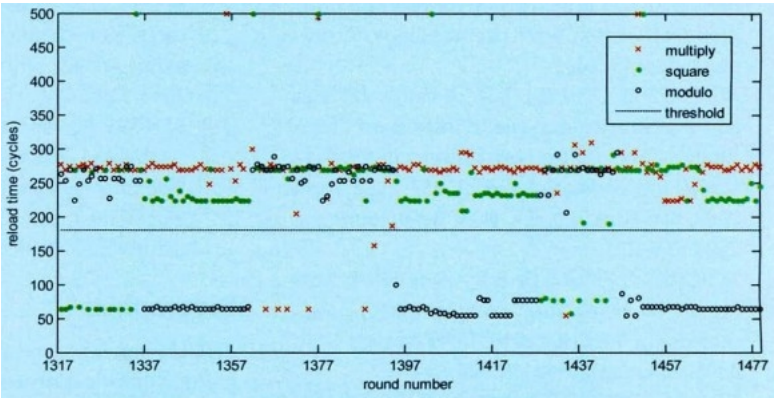


图10 重新加载时间的测量

表I 不同等待间隔与阈值的攻击成功率				
等待时间/阈值	200	180	160	120([9])
200	86%	87%	87%	
250	89%	88%	87%	
300	90%	91%	91%	
400	87%	89%	89%	
600	94%	95%	95%	
2048([9])				64%

致谢

作者感谢审稿人提出的详细审阅意见和建设性建议，这些意见有助于提高本文的质量。本研究由国家自然科学基金（项目编号：61472357、61309021、61272491、61173191）和国家重点基础研究发展计划（973计划）（项目编号：2013CB338004）资助。

参考文献

[1] KOCHER P.C. Timing Attacks Diffie-Hellman、RSA、DSS 及其他系统的实现 [C]//《密码学会议论文集1996》：1996年8月，美国加利福尼亚州圣巴巴拉。贝林：斯普林格出版社，1996年：104-113。
[2] KOCHER P、JAFJE J、JUNB.差异功率分析[C]//1999年密码学会议论文集：1999年8月，美国加利福尼亚州圣巴巴拉。柏林：斯普林格出版社，1999年：388-397。
[3] D. 缓存作为密码分析侧信道的理论应用 [R]. 技术报告

- CSTR -02-003, 布里斯托大学计算机科学系, 2002年。
- [4] 珀西瓦尔 C. 《为乐与为利而缺失的缓存》[EB/OL]. <<http://www.daemonology.net/papers/htt.pdf>>.2005 .
- [5] O. Schindler W. A RSA实现中因指令缓存分析导致的漏洞及OpenSSL上的演示[C]//CT-RSA 2008 会议论文集:2008年4月, 美国加利福尼亚州旧金山。柏林: Springer出版社, 2008:256-273。
- [6] aciicmezO、brumleyBB、grabherP.指令缓存攻击新发现[C]// CHES 2010会议录: 美国圣巴巴拉。柏林: 斯普林格出版社, 2010年: 110-124。
- [7] NEVE M, seifertj P.基于访问驱动 AES 缓存攻击研究进展[C]//SAC 2006会议录, 2006年8月, 加拿大蒙特利尔。柏林:施普林格出版社, 2007:147-162.
- [8] aciicmezO. Yet Another Micro Architectural Attack :Exploitingl-cache[C]//ACM 计算机安全体系结构研讨会论文集:2007年11月, 美国弗吉尼亚州费尔法克斯。美国纽约州纽约市:ACM, 2007:11-18。
- [9] YAROM Y, falkner K. flush + reload : 一种高 分辨率、低噪声、L3缓存侧信道攻击[EB/OL]. 加密学电子预印本档案, <http://eprint.iacr.org/>报告2013/448, 2013 .
- [10]布莱恩特·R·E、奥哈拉伦·D·R.《计算机系统:程序员视角》[M]. 北京:中国机械出版社, 2014.
- [11]《Intel 64和IA-32体系结构优化参考手册》[R]. 英特尔公司, 2012年4月。
- [12]赵淑杰、赵建勋. 信号检测与估计理论[M]. 北京:电子工业出版社, 2013.
- [13] RIVEST RL, Shamir A, Adleman L. 一种获取数字签名和公钥密码系统的方法[J]. ACM通讯, 1978,21(2):120-126.
- [14] ZHANG YJULES A,REITER M KCrossVM侧信道及其用于提取私钥的研究[C]//第19届ACM计算机与通信安全会议论文集:2012年10月,美国北卡罗来纳州罗利市。纽约,美国:ACM,2012:305-316.
- [15] ZHAO Xinjie, GUO Shize, ZHANG Fan, 等学者在《计算机与安全》2013年第39卷第4期发表的论文《基于多重推导的AES代数迹驱动缓存攻击的全面研究》中,系统阐述了此类攻击的理论框架与实施方法。
- [16] WANG Tao, ZHAO Xinjie, GUO Shize, 等. AES缓存定时模板攻击研究[J]. 中国计算机杂志, 2012, 35(2):325-341.
- [17]赵新杰、王海国等. 针对AES的驱动型缓存定时攻击[J]. 《Journal》

软件, 2011, 22(3):572-591.

- [18] YAROM Y, BENDER N.利用flush+reload缓存侧信道攻击恢复OpenSSL ECDSA 密钥[EB/OL]. 密码学ePrint档案库, <<http://eprint.iacr.org/>>报告2014/140, 2014.
- [19] BENDER N, vandePOLJ、SMARTNP 等著《哦啊.....只是一点点》: 少量旁路通道也能大有作为[C]//《CHES 会议录》2014年9月, 韩国釜山。柏林: 斯普林格出版社, 2014年: 75-92。

传记

周平, 中国石家庄市军械工程学院信息工程系博士研究生, 获中国电子科技大学学士学位, 军械工程学院硕士学位, 主要研究方向为公钥密码学的侧信道分析。
ping_01@hotmail.com

王涛, 1964年生, 1990年获中国兵器工程学院计算机应用硕士学位, 1996年获中国科学院计算技术研究所计算机应用博士学位, 现任中国兵器工程学院教授, 研究方向为信息安全与密码学。

李光, 1996年获军校炮兵指挥学院计算机工程系学士学位, 2007年获该校计算机工程系硕士学位, 现任北方电子装备研究院高级工程师, 主要研究方向为计算机安全及分组密码的侧信道分析。

张帆, 1978年生, 2012年获得美国康涅狄格大学计算机科学与工程系博士学位, 研究方向包括密码学中的侧信道分析与故障分析、计算机体系结构、无线传感器网络安全等, 现就职于浙江大学信息科学与电气工程系。*通讯作者。
电子邮箱: fanzhang@zju.edu.cn

赵新杰先后于2012年、2009年和2006年在 Ordnance Engineering 学院 Information Engineering 系获得博士学位、硕士学位和理学学士学位。现任 North Electronic 设备研究所工程师, 主要研究方向包括侧信道分析、故障分析及分组密码的综合分析。其论文荣获2012年达姆施塔特第三届 Constructive Side-Channel 分析与 Secure Design International Workshop (COSADE) 最佳论文奖。